



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

**Faculty of
Computing**

SECP3623

**Database Programming
2025/2026 SEMESTER 1**

Assignment 1 (10%)

Name	:	MOHAMED ALIF FATHI BIN ABDUL LATIF
Matric Id	:	A23CS0112
Section	:	02

Task 1: Database Creation and Integrity Constraints

Database Creation

```
8. CREATE DATABASE hostel_mgmt_alif;
9. USE hostel_mgmt_alif;
10
```

Output				
Action Output				
#	Time	Action	Message	
✓ 201	00:06:19	DROP DATABASE IF EXISTS hostel_mgmt_alif	6 row(s) affected	
✗ 202	00:06:46	USE hostel_mgmt_alif	Error Code: 1049. Unknown database 'hostel_mgmt_alif'	
✓ 203	00:07:17	CREATE DATABASE hostel_mgmt_alif	1 row(s) affected	
✓ 204	00:07:19	USE hostel_mgmt_alif	0 row(s) affected	

The CREATE DATABASE, and USE commands to create and select the database named hostel_mgmt_alif.

Table Creation: room_types

```
11. CREATE TABLE room_types (
12     type_id INT AUTO_INCREMENT PRIMARY KEY,
13     type_name ENUM('Single', 'Double', 'Premium', 'Family') NOT NULL,
14     rent DECIMAL(8,2) NOT NULL,
15     deposit DECIMAL(8,2) NOT NULL,
16     capacity INT NOT NULL
17 );
18
```

Output				
Action Output				
#	Time	Action	Message	
✗ 202	00:06:46	USE hostel_mgmt_alif	Error Code: 1049. Unknown database 'hostel_mgmt_alif'	
✓ 203	00:07:17	CREATE DATABASE hostel_mgmt_alif	1 row(s) affected	
✓ 204	00:07:19	USE hostel_mgmt_alif	0 row(s) affected	
✓ 205	00:08:02	CREATE TABLE room_types (type_id INT AUTO_INCREMENT PRIMARY KEY, type_name ENUM('Single', 'D...	0 row(s) affected	

The CREATE TABLE room_types statement with type_id as PRIMARY KEY, type_name, rent, deposit, and capacity as NOT NULL columns.

Table Creation: rooms

```
19 • CREATE TABLE rooms (  
20     room_id INT AUTO_INCREMENT PRIMARY KEY,  
21     type_id INT NOT NULL,  
22     room_no VARCHAR(10) NOT NULL UNIQUE,  
23     floor_no INT NOT NULL,  
24     is_occupied BOOLEAN DEFAULT FALSE NOT NULL,  
25     FOREIGN KEY (type_id) REFERENCES room_types(type_id)  
26 );  
27
```

Output			
Action Output			
#	Time	Action	Message
203	00:07:17	CREATE DATABASE hostel_mgmt_alif	1 row(s) affected
204	00:07:19	USE hostel_mgmt_alif	0 row(s) affected
205	00:08:02	CREATE TABLE room_types (type_id INT AUTO_INCREMENT PRIMARY KEY, type_name ENUM('Single', 'D...	0 row(s) affected
206	00:09:02	CREATE TABLE rooms (room_id INT AUTO_INCREMENT PRIMARY KEY, type_id INT NOT NULL, room_n...	0 row(s) affected

The CREATE TABLE rooms statement with room_id as PRIMARY KEY, type_id as FOREIGN KEY referencing room_types, and room_no as UNIQUE and NOT NULL.

Table Creation: students

```
28 • CREATE TABLE students (  
29     student_id INT AUTO_INCREMENT PRIMARY KEY,  
30     room_id INT NOT NULL,  
31     fname VARCHAR(50) NOT NULL,  
32     lname VARCHAR(50) NOT NULL,  
33     status ENUM('ACTIVE', 'NON_ACTIVE') NOT NULL,  
34     checkin_date DATE NOT NULL,  
35     FOREIGN KEY (room_id) REFERENCES rooms(room_id)  
36 );
```

Output			
Action Output			
#	Time	Action	Message
204	00:07:19	USE hostel_mgmt_alif	0 row(s) affected
205	00:08:02	CREATE TABLE room_types (type_id INT AUTO_INCREMENT PRIMARY KEY, type_name ENUM('Single', 'D...	0 row(s) affected
206	00:09:02	CREATE TABLE rooms (room_id INT AUTO_INCREMENT PRIMARY KEY, type_id INT NOT NULL, room_n...	0 row(s) affected
207	00:13:30	CREATE TABLE students (student_id INT AUTO_INCREMENT PRIMARY KEY, room_id INT NOT NULL, fn...	0 row(s) affected

The CREATE TABLE students statement with student_id as PRIMARY KEY and room_id as FOREIGN KEY referencing rooms.

Table Creation: maintenance

```
37
38 • CREATE TABLE maintenance (
39     maint_id INT AUTO_INCREMENT PRIMARY KEY,
40     room_id INT NOT NULL,
41     issue_desc TEXT NOT NULL,
42     severity ENUM('LOW', 'MEDIUM', 'HIGH') NOT NULL,
43     status ENUM('OPEN', 'RESOLVED') NOT NULL,
44     reported_on DATE NOT NULL,
45     resolved_on DATE,
46     FOREIGN KEY (room_id) REFERENCES rooms(room_id)
47 );
48
```

Output				
Action Output				
#	Time	Action	Message	
213	00:16:58	CREATE TABLE room_types (type_id INT AUTO_INCREMENT PRIMARY KEY, type_name ENUM('Single', 'D...	0 row(s) affected	
214	00:17:00	CREATE TABLE rooms (room_id INT AUTO_INCREMENT PRIMARY KEY, type_id INT NOT NULL, room_n...	0 row(s) affected	
215	00:17:02	CREATE TABLE students (student_id INT AUTO_INCREMENT PRIMARY KEY, room_id INT NOT NULL, fn...	0 row(s) affected	
216	00:17:04	CREATE TABLE maintenance (maint_id INT AUTO_INCREMENT PRIMARY KEY, room_id INT NOT NULL, ...	0 row(s) affected	

The CREATE TABLE maintenance statement with maint_id as PRIMARY KEY and room_id as FOREIGN KEY referencing rooms.

Table Creation: payments

```
49 • CREATE TABLE payments (
50     payment_id INT AUTO_INCREMENT PRIMARY KEY,
51     student_id INT NOT NULL,
52     amount DECIMAL(8,2) NOT NULL,
53     paid_on DATE NOT NULL,
54     method ENUM('CASH', 'FPX', 'CARD', 'TNG') NOT NULL,
55     note TEXT,
56     FOREIGN KEY (student_id) REFERENCES students(student_id)
57 );
58
```

Output				
Action Output				
#	Time	Action	Message	
214	00:17:00	CREATE TABLE rooms (room_id INT AUTO_INCREMENT PRIMARY KEY, type_id INT NOT NULL, room_n...	0 row(s) affected	
215	00:17:02	CREATE TABLE students (student_id INT AUTO_INCREMENT PRIMARY KEY, room_id INT NOT NULL, fn...	0 row(s) affected	
216	00:17:04	CREATE TABLE maintenance (maint_id INT AUTO_INCREMENT PRIMARY KEY, room_id INT NOT NULL, ...	0 row(s) affected	
217	00:18:48	CREATE TABLE payments (payment_id INT AUTO_INCREMENT PRIMARY KEY, student_id INT NOT NULL, ...	0 row(s) affected	

The CREATE TABLE payments statement with payment_id as PRIMARY KEY and student_id as FOREIGN KEY referencing students.

ALTER TABLE (Add UNIQUE email)

```
59 # Alter table, create and delete test table
60 ALTER TABLE students ADD COLUMN email VARCHAR(50) UNIQUE;
61
```

Output				
Action Output				
#	Time	Action	Message	
215	00:17:02	CREATE TABLE students (student_id INT AUTO_INCREMENT PRIMARY KEY, room_id INT NOT NULL, fn...	0 row(s) affected	
216	00:17:04	CREATE TABLE maintenance (maint_id INT AUTO_INCREMENT PRIMARY KEY, room_id INT NOT NULL, ...	0 row(s) affected	
217	00:18:48	CREATE TABLE payments (payment_id INT AUTO_INCREMENT PRIMARY KEY, student_id INT NOT NULL, ...	0 row(s) affected	
218	00:19:34	ALTER TABLE students ADD COLUMN email VARCHAR(50) UNIQUE	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	

The ALTER TABLE students ADD COLUMN email VARCHAR(50) UNIQUE command to add the email column and set its UNIQUE constraint.

DROP TABLE Demonstration

```
62 CREATE TABLE test (
63     id INT AUTO_INCREMENT PRIMARY KEY
64 );
65
66 DROP TABLE test;
67
```

Output				
Action Output				
#	Time	Action	Message	
217	00:18:48	CREATE TABLE payments (payment_id INT AUTO_INCREMENT PRIMARY KEY, student_id INT NOT NULL, ...	0 row(s) affected	
218	00:19:34	ALTER TABLE students ADD COLUMN email VARCHAR(50) UNIQUE	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	
219	00:19:50	CREATE TABLE test (id INT AUTO_INCREMENT PRIMARY KEY)	0 row(s) affected	
220	00:19:51	DROP TABLE test	0 row(s) affected	

The CREATE TABLE test and DROP TABLE test commands demonstrating schema maintenance by safely creating and dropping a temporary table.

Task 2: Data Manipulation and Filtering

Data Insertion: room_types

```
72 • INSERT INTO room_types (type_name, rent, deposit, capacity) VALUES
73 ('Single', 700.00, 300.00, 1),
74 ('Single', 750.00, 300.00, 1),
75 ('Single', 800.00, 300.00, 1),
76 ('Double', 500.00, 200.00, 2),
77 ('Double', 550.00, 200.00, 2),
78 ('Double', 600.00, 200.00, 2),
79 ('Premium', 700.00, 300.00, 2),
80 ('Premium', 750.00, 300.00, 2),
81 ('Premium', 800.00, 300.00, 2),
82 ('Family', 1200.00, 500.00, 4),
83 ('Family', 1250.00, 500.00, 4),
84 ('Family', 1300.00, 500.00, 4);
85
```

Output			
Action Output			
#	Time	Action	Message
218	00:19:34	ALTER TABLE students ADD COLUMN email VARCHAR(50) UNIQUE	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
219	00:19:50	CREATE TABLE test (id INT AUTO_INCREMENT PRIMARY KEY)	0 row(s) affected
220	00:19:51	DROP TABLE test	0 row(s) affected
221	00:21:53	INSERT INTO room_types (type_name, rent, deposit, capacity) VALUES ('Single', 700.00, 300.00, 1), ('Single', 750.00, 300.00, 1), ('Single', 800.00, 300.00, 1), ('Double', 500.00, 200.00, 2), ('Double', 550.00, 200.00, 2), ('Double', 600.00, 200.00, 2), ('Premium', 700.00, 300.00, 2), ('Premium', 750.00, 300.00, 2), ('Premium', 800.00, 300.00, 2), ('Family', 1200.00, 500.00, 4), ('Family', 1250.00, 500.00, 4), ('Family', 1300.00, 500.00, 4);	12 row(s) affected Records: 12 Duplicates: 0 Warnings: 0

The INSERT INTO room_types statement to populate the table with different rent, deposit, and capacity records.

Data Insertion: rooms

```
86 • INSERT INTO rooms (type_id, room_no, floor_no) VALUES
87 (4, 'A101', 1),
88 (4, 'A102', 1),
89 (5, 'A103', 1),
90 (5, 'A104', 1),
91 (6, 'A105', 1),
92 (1, 'A201', 2),
93 (1, 'A202', 2),
94 (2, 'A203', 2),
95 (2, 'A204', 2),
96 (3, 'A205', 2),
97 (7, 'A301', 3),
98 (8, 'A302', 3),
99 (9, 'A303', 3),
100 (10, 'A401', 4),
101 (11, 'A402', 4),
102 (12, 'A403', 4);
```

Output				
Action Output				
#	Time	Action	Message	
✓ 219	00:19:50	CREATE TABLE test (id INT AUTO_INCREMENT PRIMARY KEY)	0 row(s) affected	
✓ 220	00:19:51	DROP TABLE test	0 row(s) affected	
✓ 221	00:21:53	INSERT INTO room_types (type_name, rent, deposit, capacity) VALUES ('Single', 700.00, 300.00, 1), ('Single', 750.00, 350.00, 1), ('Double', 1200.00, 600.00, 2), ('Double', 1300.00, 650.00, 2), ('Suite', 2000.00, 1000.00, 3), ('Suite', 2200.00, 1100.00, 3), ('Presidential', 5000.00, 2500.00, 5), ('Presidential', 5500.00, 2750.00, 5)	12 row(s) affected Records: 12 Duplicates: 0 Warnings: 0	
✓ 222	00:22:20	INSERT INTO rooms (type_id, room_no, floor_no) VALUES (4, 'A101', 1), (4, 'A102', 1), (5, 'A103', 1), (5, 'A104', 1), (6, 'A105', 1), (1, 'A201', 2), (1, 'A202', 2), (2, 'A203', 2), (2, 'A204', 2), (3, 'A205', 2), (7, 'A301', 3), (8, 'A302', 3), (9, 'A303', 3), (10, 'A401', 4), (11, 'A402', 4), (12, 'A403', 4)	16 row(s) affected Records: 16 Duplicates: 0 Warnings: 0	

The INSERT INTO rooms statement to populate the table distributed across a minimum of three floors, linked to valid room types.

Data Insertion: students

```
104 • INSERT INTO students (room_id, fname, lname, status, checkin_date, email) VALUES
105 (1,'Alif','Fathi','ACTIVE','2025-09-26','alif@graduate.utm.my'),
106 (1,'Sabrina','Carpenter','ACTIVE','2025-10-08','sabrina@graduate.utm.my'),
107 (1,'Rie','Takahashi','NON_ACTIVE','2025-08-08','rie@graduate.utm.my'),
108 (2,'Iman','Abadi','ACTIVE','2025-09-10','iman@graduate.utm.my'),
109 (2,'Bella','Astillah','ACTIVE','2025-09-01','bella@graduate.utm.my'),
110 (3,'Adam','Ramli','ACTIVE','2025-10-01','adam@graduate.utm.my'),
111 (3,'Cindy','Liew','NON_ACTIVE','2025-08-15','cindy@graduate.utm.my'),
112 (4,'Siti','Farah','ACTIVE','2025-09-05','siti@graduate.utm.my'),
113 (4,'Naim','Daniel','NON_ACTIVE','2025-08-10','naim@graduate.utm.my'),
114 (5,'Emma','Watson','ACTIVE','2025-10-11','emma@graduate.utm.my'),
115 (5,'Elizabeth','Tan','ACTIVE','2025-10-15','elizabeth@graduate.utm.my'),
116 (6,'Ahmad','Afliq','NON_ACTIVE','2025-08-22','ahmad@graduate.utm.my'),
117 (7,'Farid','Hassan','ACTIVE','2025-10-20','farid@graduate.utm.my'),
118 (8,'Nadia','Zainal','ACTIVE','2025-09-25','nadia@graduate.utm.my'),
119 (9,'Hafiz','Ali','NON_ACTIVE','2025-08-30','hafiz@graduate.utm.my'),
120 (10,'Amirah','Osman','ACTIVE','2025-10-05','amirah@graduate.utm.my'),
121 (11,'Danish','Halim','ACTIVE','2025-09-12','danish@graduate.utm.my'),
122 (11,'Natsuki','Deguchi','ACTIVE','2025-09-28','natsuki@graduate.utm.my'),
123 (12,'Justin','Bieber','ACTIVE','2025-09-08','justin@graduate.utm.my'),
124 (12,'Iqmal','Hakim','NON_ACTIVE','2025-08-03','iqmal@graduate.utm.my'),
125 (13,'Damiya','Aina','NON_ACTIVE','2025-08-03','damiya@graduate.utm.my'),
126 (14,'Janna','Nick','ACTIVE','2025-10-02','janna@graduate.utm.my'),
127 (14,'Nur','Firzana','ACTIVE','2025-10-02','nur@graduate.utm.my'),
128 (14,'Nurul','Adriana','ACTIVE','2025-10-02','nurul@graduate.utm.my'),
129 (15,'Adib','Zikri','ACTIVE','2025-10-03','adib@graduate.utm.my'),
130 (16,'Furina','Focalors','NON_ACTIVE','2025-08-06','furina@graduate.utm.my');
131
```

Output

#	Time	Action	Message
✓ 220	00:19:51	DROP TABLE test	0 row(s) affected
✓ 221	00:21:53	INSERT INTO room_types (type_name, rent, deposit, capacity) VALUES ('Single', 700.00, 300.00, 1), ('Single', 750.0...	12 row(s) affected Records: 12 Duplicates: 0 Warnings: 0
✓ 222	00:22:20	INSERT INTO rooms (type_id, room_no, floor_no) VALUES (4, 'A101', 1), (4, 'A102', 1), (5, 'A103', 1), (5, 'A104', 1), (6,...	16 row(s) affected Records: 16 Duplicates: 0 Warnings: 0
✓ 223	00:22:48	INSERT INTO students (room_id, fname, lname, status, checkin_date, email) VALUES (1,'Alif','Fathi','ACTIVE','2025-0...	26 row(s) affected Records: 26 Duplicates: 0 Warnings: 0

The INSERT INTO students statement to populate the table includes 'ACTIVE' and 'NON_ACTIVE' statuses, with at least one student name starting with 'A'.

Data Insertion: maintenance

```
123 INSERT INTO maintenance (room_id, issue_desc, severity, status, reported_on, resolved_on) VALUES
124 ('Bed slat broken','MEDIUM','OPEN','2025-10-26',NULL),
125 ('Lamp flickering','LOW','RESOLVED','2025-09-05','2025-09-08'),
126 ('Floor crack','HIGH','OPEN','2025-10-12',NULL),
127 ('Locker door loose','LOW','RESOLVED','2025-08-20','2025-08-25'),
128 ('Ceiling fan wobbling','MEDIUM','OPEN','2025-10-20',NULL),
129 ('Bulb blown','LOW','RESOLVED','2025-09-09','2025-09-10'),
130 ('Window latch broken','HIGH','OPEN','2025-10-22',NULL),
131 ('Door hinge loose','LOW','RESOLVED','2025-08-10','2025-08-14'),
132 ('Drawer jammed','LOW','OPEN','2025-10-25',NULL),
133 ('Chair leg broken','MEDIUM','OPEN','2025-10-27',NULL),
134 ('Window glass cracked','MEDIUM','OPEN','2025-10-24',NULL),
135 ('Power socket loose','HIGH','OPEN','2025-09-30',NULL),
136 ('Door handle loose','LOW','RESOLVED','2025-08-05','2025-08-14'),
137 ('Floor tile cracked','MEDIUM','RESOLVED','2025-08-06','2025-08-12'),
138 ('Shower pipe burst','HIGH','OPEN','2025-10-27',NULL),
139 ('Curtain rod bent','MEDIUM','RESOLVED','2025-08-10','2025-08-15'),
140 ('Lamp wiring exposed','HIGH','RESOLVED','2025-09-20','2025-09-22'),
141 ('Sink leaking','LOW','OPEN','2025-10-23',NULL),
142 ('Mirror cracked','HIGH','OPEN','2025-10-28',NULL),
143 ('Lamp switch loose','LOW','RESOLVED','2025-09-12','2025-09-13'),
144 ('Wall paint peeling','LOW','RESOLVED','2025-09-12','2025-09-20'),
145 ('Power socket intermittent','MEDIUM','OPEN','2025-10-10',NULL),
146 ('Bed frame wobbling','MEDIUM','OPEN','2025-10-27',NULL),
147 ('Power socket loose','MEDIUM','OPEN','2025-10-29',NULL),
148 ('Toilet flush faulty','HIGH','OPEN','2025-10-29',NULL),
149 ('Locker door dented','LOW','RESOLVED','2025-08-18','2025-08-22'),
150 ('Desk drawer stuck','LOW','RESOLVED','2025-09-21','2025-09-23')
```

#	Time	Action	Message
✓ 221	00:21:53	INSERT INTO room_types (type_name, rent, deposit, capacity) VALUES ('Single', 700.00, 300.00, 1), ('Single', 750.00, 300.00, 1)	12 row(s) affected Records: 12 Duplicates: 0 Warnings: 0
✓ 222	00:22:20	INSERT INTO rooms (type_id, room_no, floor_no) VALUES (4, 'A101', 1), (4, 'A102', 1), (5, 'A103', 1), (5, 'A104', 1), (6, 'B101', 1), (6, 'B102', 1), (6, 'B103', 1), (6, 'B104', 1), (6, 'C101', 1), (6, 'C102', 1), (6, 'C103', 1), (6, 'C104', 1)	16 row(s) affected Records: 16 Duplicates: 0 Warnings: 0
✓ 223	00:22:48	INSERT INTO students (room_id, fname, lname, status, checkin_date, email) VALUES (1, 'Ali', 'Fathi', 'ACTIVE', '2025-01-01', 'ali.fathi@university.edu'), (2, 'Fatma', 'Abdullah', 'ACTIVE', '2025-01-01', 'fatma.abdullah@university.edu'), (3, 'Omar', 'Mohammed', 'ACTIVE', '2025-01-01', 'omar.mohammed@university.edu'), (4, 'Nora', 'Ahmed', 'ACTIVE', '2025-01-01', 'nora.ahmed@university.edu'), (5, 'Youssef', 'Sara', 'ACTIVE', '2025-01-01', 'youssef.sara@university.edu'), (6, 'Layla', 'Hassan', 'ACTIVE', '2025-01-01', 'layla.hassan@university.edu'), (7, 'Mehdi', 'Khalid', 'ACTIVE', '2025-01-01', 'mehdi.khalid@university.edu'), (8, 'Amr', 'Eman', 'ACTIVE', '2025-01-01', 'amr.eman@university.edu'), (9, 'Sofia', 'Ibrahim', 'ACTIVE', '2025-01-01', 'sofia.ibrahim@university.edu'), (10, 'Adnan', 'Rana', 'ACTIVE', '2025-01-01', 'adnan.rana@university.edu')	26 row(s) affected Records: 26 Duplicates: 0 Warnings: 0
✓ 224	00:23:18	INSERT INTO maintenance (room_id, issue_desc, severity, status, reported_on, resolved_on) VALUES (1, 'Bed slat br...', 'Medium', 'PENDING', '2025-01-01', NULL), (2, 'Water leak under sink', 'High', 'PENDING', '2025-01-01', NULL), (3, 'Broken light fixture', 'Low', 'PENDING', '2025-01-01', NULL), (4, 'Door handle loose', 'Low', 'PENDING', '2025-01-01', NULL), (5, 'Air conditioning not working', 'High', 'PENDING', '2025-01-01', NULL), (6, 'Roof leak in hallway', 'High', 'PENDING', '2025-01-01', NULL), (7, 'Plumbing issue in bathroom', 'Medium', 'PENDING', '2025-01-01', NULL), (8, 'Crack in wall', 'Low', 'PENDING', '2025-01-01', NULL), (9, 'Stained carpet', 'Low', 'PENDING', '2025-01-01', NULL), (10, 'Faulty electrical outlet', 'Medium', 'PENDING', '2025-01-01', NULL)	28 row(s) affected Records: 28 Duplicates: 0 Warnings: 0

The INSERT INTO maintenance statement to populate the table includes diverse severity (LOW, MEDIUM, HIGH) and status (OPEN, RESOLVED) levels.

Data Insertion: payments

```
162 • INSERT INTO payments (student_id, amount, paid_on, method, note) VALUES
```

```
163 (1,500.00,'2025-10-01','CASH','Rent (Oct 2025)'),
164 (2,500.00,'2025-11-01','FPX','Rent (Nov 2025)'),
165 (3,500.00,'2025-09-01','CARD','Rent (Sep 2025)'),
166 (4,500.00,'2025-10-01','TNG','Rent (Oct 2025)'),
167 (5,500.00,'2025-10-01','CASH','Rent (Oct 2025)'),
168 (6,550.00,'2025-11-01','FPX','Rent (Nov 2025)'),
169 (7,550.00,'2025-09-01','CARD','Rent (Sep 2025)'),
170 (8,550.00,'2025-10-01','TNG','Rent (Oct 2025)'),
171 (9,550.00,'2025-09-01','CASH','Rent (Sep 2025)'),
172 (10,600.00,'2025-11-01','FPX','Rent (Nov 2025)'),
173 (11,600.00,'2025-11-01','CARD','Rent (Nov 2025)'),
174 (12,700.00,'2025-09-01','TNG','Rent (Sep 2025)'),
175 (13,700.00,'2025-11-01','CASH','Rent (Nov 2025)'),
176 (14,750.00,'2025-10-01','FPX','Rent (Oct 2025)'),
177 (15,750.00,'2025-09-01','CARD','Rent (Sep 2025)'),
178 (16,800.00,'2025-11-01','TNG','Rent (Nov 2025)'),
179 (17,700.00,'2025-10-01','CASH','Rent (Oct 2025)'),
180 (18,700.00,'2025-10-01','FPX','Rent (Oct 2025)'),
181 (19,750.00,'2025-10-01','CARD','Rent (Oct 2025)'),
182 (20,750.00,'2025-09-01','TNG','Rent (Sep 2025)'),
183 (21,800.00,'2025-09-01','CASH','Rent (Sep 2025)'),
184 (22,1200.00,'2025-11-01','FPX','Rent (Nov 2025)'),
185 (23,1200.00,'2025-11-01','CARD','Rent (Nov 2025)'),
186 (24,1200.00,'2025-11-01','CASH','Rent (Nov 2025)'),
187 (25,1250.00,'2025-11-01','FPX','Rent (Nov 2025)'),
188 (26,1300.00,'2025-09-01','CARD','Rent (Sep 2025));
```

Output				
Action Output				
#	Time	Action	Message	
✓ 222	00:22:20	INSERT INTO rooms (type_id, room_no, floor_no) VALUES (4, 'A101', 1), (4, 'A102', 1), (5, 'A103', 1), (5, 'A104', 1), (6, ...	16 row(s) affected Records: 16 Duplicates: 0 Warnings: 0	
✓ 223	00:22:48	INSERT INTO students (room_id, fname, lname, status, checkin_date, email) VALUES (1,'Aif','Fathi','ACTIVE','2025-0...	26 row(s) affected Records: 26 Duplicates: 0 Warnings: 0	
✓ 224	00:23:18	INSERT INTO maintenance (room_id, issue_desc, severity, status, reported_on, resolved_on) VALUES (1,'Bed slat br...	28 row(s) affected Records: 28 Duplicates: 0 Warnings: 0	
✓ 225	00:23:36	INSERT INTO payments (student_id, amount, paid_on, method, note) VALUES (1,500.00,'2025-10-01','CASH','Rent (...	26 row(s) affected Records: 26 Duplicates: 0 Warnings: 0	

The INSERT INTO payments statement to populate the table, covering at least two months and two payment methods, linked to existing students.

UPDATE Operation (is_occupied)

```
194 • UPDATE rooms
195 SET is_occupied = CASE
196     WHEN room_id IN (SELECT room_id FROM students WHERE status = 'ACTIVE') THEN TRUE
197     ELSE FALSE
198 END;
199
```

Output				
Action Output				
#	Time	Action	Message	
✓ 15	00:24:34	INSERT INTO maintenance (room_id, issue_desc, severity, status, reported_on, resolved_on) VALUES (1,'Bed slat bro...	28 row(s) affected Records: 28 Duplicates: 0 Warnings: 0	
✓ 16	00:24:57	INSERT INTO payments (student_id, amount, paid_on, method, note) VALUES (1,500.00,'2025-10-01','CASH','Rent (...	26 row(s) affected Records: 26 Duplicates: 0 Warnings: 0	
✓ 17	00:25:23	SET SQL_SAFE_UPDATES = 0	0 row(s) affected	
✓ 18	00:25:27	UPDATE rooms SET is_occupied = CASE WHEN room_id IN (SELECT room_id FROM students WHERE status = '...	12 row(s) affected Rows matched: 16 Changed: 12 Warnings: 0	

The UPDATE rooms command to set is_occupied = TRUE for rooms with 'ACTIVE' students and FALSE otherwise.

DELETE Operation (Old Resolved Maintenance)

```
200 • DELETE FROM maintenance
201 WHERE status='RESOLVED' AND DATEDIFF(CURDATE(), reported_on) > 60;
202
```

Output

#	Time	Action	Message
16	00:24:57	INSERT INTO payments (student_id, amount, paid_on, method, note) VALUES (1,500.00,'2025-10-01','CASH','Rent (...)	26 row(s) affected Records: 26 Duplicates: 0 Warnings: 0
17	00:25:23	SET SQL_SAFE_UPDATES = 0	0 row(s) affected
18	00:25:27	UPDATE rooms SET is_occupied = CASE WHEN room_id IN (SELECT room_id FROM students WHERE status = '...)	12 row(s) affected Rows matched: 16 Changed: 12 Warnings: 0
19	00:27:28	DELETE FROM maintenance WHERE status='RESOLVED' AND DATEDIFF(CURDATE(), reported_on) > 60	11 row(s) affected

The DELETE FROM maintenance command to remove maintenance records where the status is 'RESOLVED' and the reported_on date is older than 60 days.

Filtering Query using BETWEEN

```
205 • SELECT * FROM room_types WHERE rent BETWEEN 400 AND 800;
```

Result Grid

type_id	type_name	rent	deposit	capacity
1	Single	700.00	300.00	1
2	Single	750.00	300.00	1
3	Single	800.00	300.00	1
4	Double	500.00	200.00	2
5	Double	550.00	200.00	2
6	Double	600.00	200.00	2
7	Premium	700.00	300.00	2
8	Premium	750.00	300.00	2
9	Premium	800.00	300.00	2

room_types 1 x

Output

#	Time	Action	Message
19	00:27:28	DELETE FROM maintenance WHERE status='RESOLVED' AND DATEDIFF(CURDATE(), reported_on) > 60	11 row(s) affected
20	00:27:45	SELECT * FROM room_types WHERE rent BETWEEN 400 AND 800 LIMIT 0, 1000	9 row(s) returned

The SELECT query using WHERE rent BETWEEN 400 AND 800 to display room types with rent in the specified range.

Filtering Query using LIKE

207 • **SELECT * FROM students WHERE fname LIKE 'A%';**

208

student_id	room_id	fname	lname	status	checkin_date	email
1	1	Alif	Fathi	ACTIVE	2025-09-26	alif@graduate.utm.my
6	3	Adam	Ramli	ACTIVE	2025-10-01	adam@graduate.utm.my
12	6	Ahmad	Afiq	NON_ACTIVE	2025-08-22	ahmad@graduate.utm.my
16	10	Amirah	Osman	ACTIVE	2025-10-05	amirah@graduate.utm.my
25	15	Adib	Zikri	ACTIVE	2025-10-03	adib@graduate.utm.my

students 2 x

Output

Action Output

#	Time	Action	Message
20	00:27:45	SELECT * FROM room_types WHERE rent BETWEEN 400 AND 800 LIMIT 0, 1000	9 row(s) returned
21	00:28:19	SELECT * FROM students WHERE fname LIKE 'A%' LIMIT 0, 1000	5 row(s) returned

The SELECT query using WHERE fname LIKE 'A%' to retrieve students whose first name starts with the letter 'A'.

Filtering Query using IN

209 • **SELECT * FROM payments WHERE method IN ('FPX','CARD');**

210

payment_id	student_id	amount	paid_on	method	note
2	2	500.00	2025-11-01	FPX	Rent (Nov 2025)
3	3	500.00	2025-09-01	CARD	Rent (Sep 2025)
6	6	550.00	2025-11-01	FPX	Rent (Nov 2025)
7	7	550.00	2025-09-01	CARD	Rent (Sep 2025)
10	10	600.00	2025-11-01	FPX	Rent (Nov 2025)
11	11	600.00	2025-11-01	CARD	Rent (Nov 2025)
14	14	750.00	2025-10-01	FPX	Rent (Oct 2025)
15	15	750.00	2025-09-01	CARD	Rent (Sep 2025)
18	18	700.00	2025-10-01	FPX	Rent (Oct 2025)
19	19	750.00	2025-10-01	CARD	Rent (Oct 2025)
22	22	1200.00	2025-11-01	FPX	Rent (Nov 2025)
23	23	1200.00	2025-11-01	CARD	Rent (Nov 2025)
25	25	1250.00	2025-11-01	FPX	Rent (Nov 2025)
26	26	1300.00	2025-09-01	CARD	Rent (Sep 2025)

payments 3 x

Output

Action Output

#	Time	Action	Message
21	00:28:19	SELECT * FROM students WHERE fname LIKE 'A%' LIMIT 0, 1000	5 row(s) returned
22	00:28:37	SELECT * FROM payments WHERE method IN ('FPX','CARD') LIMIT 0, 1000	14 row(s) returned

The SELECT query using WHERE method IN ('FPX','CARD') to display payments made using FPX or CARD methods.

Filtering Query (AND, OR, and NOT)

```
211 SELECT *
212 FROM students
213 WHERE (status = 'ACTIVE' OR status = 'NON_ACTIVE') AND NOT (email LIKE '%gmail.com');
```

student_id	room_id	fname	lname	status	checkin_date	email
1	1	Alif	Fathi	ACTIVE	2025-09-26	alif@graduate.utm.my
2	1	Sabrina	Carpenter	ACTIVE	2025-10-08	sabrina@graduate.utm.my
3	1	Rie	Takahashi	NON_ACTIVE	2025-08-08	rie@graduate.utm.my
4	2	Iman	Abadi	ACTIVE	2025-09-10	iman@graduate.utm.my
5	2	Bella	Astillah	ACTIVE	2025-09-01	bella@graduate.utm.my
6	3	Adam	Ramli	ACTIVE	2025-10-01	adam@graduate.utm.my
7	3	Cindy	Liew	NON_ACTIVE	2025-08-15	cindy@graduate.utm.my
8	4	Siti	Farah	ACTIVE	2025-09-05	siti@graduate.utm.my
9	4	Naim	Daniel	NON_ACTIVE	2025-08-10	naim@graduate.utm.my
10	5	Emma	Watson	ACTIVE	2025-10-11	emma@graduate.utm.my
11	5	Elizabeth	Tan	ACTIVE	2025-10-15	elizabeth@graduate.utm.my
12	6	Ahmad	Afiq	NON_ACTIVE	2025-08-22	ahmad@graduate.utm.my
13	7	Farid	Hassan	ACTIVE	2025-10-20	farid@graduate.utm.my
14	8	Nadia	Zainal	ACTIVE	2025-09-25	nadia@graduate.utm.my
15	9	Hafiz	Ali	NON_ACTIVE	2025-08-30	hafiz@graduate.utm.my

students 4 x

Output

Action Output

#	Time	Action	Message
22	00:28:37	SELECT * FROM payments WHERE method IN ('FPX','CARD') LIMIT 0, 1000	14 row(s) returned
23	00:30:29	SELECT * FROM students WHERE (status = 'ACTIVE' OR status = 'NON_ACTIVE') AND NOT (email LIKE '%gmail.co...)	26 row(s) returned

The SELECT query combines AND, OR, and NOT logical operators to filter for all students (ACTIVE or NON_ACTIVE) whose email address is NOT a standard Gmail account.

Aggregate Function (COUNT)

```
217 # Aggregate Function
218 SELECT room_id, COUNT(student_id) AS total_students
219 FROM students
220 GROUP BY room_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	room_id	total_students
▶	1	3
	2	2
	3	2
	4	2
	5	2
	6	1
	7	1
	8	1
	9	1
	10	1
	11	2
	12	2
	13	1
	14	3
	15	1

Result 5 x

Output

Action Output

#	Time	Action	Message
✓ 23	00:30:29	SELECT * FROM students WHERE (status = 'ACTIVE' OR status = 'NON_ACTIVE') AND NOT (email LIKE '%gmail.co...)	26 row(s) returned
✓ 24	00:30:59	SELECT room_id, COUNT(student_id) AS total_students FROM students GROUP BY room_id LIMIT 0, 1000	16 row(s) returned

The SELECT query using the COUNT() aggregate function to display the total number of students per room.

String Functions (CONCAT and UPPER)

```
222 # String Function
223 SELECT student_id, CONCAT(UPPER(fname), ' ', UPPER(lname)) AS full_name_upper
224 FROM students;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [I](#)

student_id	full_name_upper
1	ALIF FATHI
2	SABRINA CARPENTER
3	RIE TAKAHASHI
4	IMAN ABADI
5	BELLA ASTILLAH
6	ADAM RAMLI
7	CINDY LIEW
8	SITI FARAH
9	NAIM DANIEL
10	EMMA WATSON
11	ELIZABETH TAN
12	AHMAD AFIQ
13	FARID HASSAN
14	NADIA ZAINAL
15	HAFIZ ALI

Result 6 x

Output

Action Output

#	Time	Action	Message
24	00:30:59	SELECT room_id, COUNT(student_id) AS total_students FROM students GROUP BY room_id LIMIT 0, 1000	16 row(s) returned
25	00:31:27	SELECT student_id, CONCAT(UPPER(fname), ' ', UPPER(lname)) AS full_name_upper FROM students LIMIT 0, 1000	26 row(s) returned

The SELECT query using the CONCAT() and UPPER() string functions to display the student's full name in uppercase.

Task 3: Reporting and Aggregation

View Creation (v_room_status)

```
230 CREATE VIEW v_room_status AS
231 SELECT
232     r.room_no,
233     rt.type_name,
234     rt.rent,
235     r.floor_no,
236     rt.capacity,
237     CASE WHEN s.n_occupants IS NULL THEN 0 ELSE s.n_occupants END AS n_occupants,
238     CASE WHEN m.pending_issues IS NULL THEN 0 ELSE m.pending_issues END AS pending_issues,
239     NOT r.is_occupied AS is_vacant
240 FROM rooms r
241 JOIN room_types rt ON r.type_id = rt.type_id
242 LEFT JOIN (
243     SELECT room_id, SUM(status='ACTIVE') AS n_occupants
244     FROM students
245     GROUP BY room_id
246 ) s ON r.room_id = s.room_id
247 LEFT JOIN (
248     SELECT room_id, SUM(status='OPEN') AS pending_issues
249     FROM maintenance
250     GROUP BY room_id
251 ) m ON r.room_id = m.room_id;
```

Output

Action Output

#	Time	Action	Message
25	00:31:27	SELECT student_id, CONCAT(UPPER(fname), ' ', UPPER(lname)) AS full_name_upper FROM students LIMIT 0, 1000	26 row(s) returned
26	00:31:54	CREATE VIEW v_room_status AS SELECT r.room_no, rt.type_name, rt.rent, r.floor_no, rt.capacity, C...	0 row(s) affected

The CREATE VIEW v_room_status statement, which calculates n_occupants (active student count), pending_issues (open maintenance count), and is_vacant for each room.

View Retrieval

253 **SELECT * FROM v_room_status;**

room_no	type_name	rent	floor_no	capacity	n_occupants	pending_issues	is_vacant
A201	Single	700.00	2	1	0	2	1
A202	Single	700.00	2	1	1	0	0
A203	Single	750.00	2	1	1	1	0
A204	Single	750.00	2	1	0	0	1
A205	Single	800.00	2	1	1	2	0
A101	Double	500.00	1	2	2	1	0
A102	Double	500.00	1	2	2	1	0
A103	Double	550.00	1	2	1	1	0
A104	Double	550.00	1	2	1	1	0
A105	Double	600.00	1	2	2	2	0
A301	Premium	700.00	3	2	2	0	0
A302	Premium	750.00	3	2	1	1	0
A303	Premium	800.00	3	2	0	2	1
A401	Family	1200.00	4	4	3	1	0
A402	Family	1250.00	4	4	1	0	0
A403	Family	1300.00	4	4	0	1	1

Output

#	Time	Action	Message
26	00:31:54	CREATE VIEW v_room_status AS SELECT r.room_no, rt.type_name, rt.rent, r.floor_no, rt.capacity, C...	0 row(s) affected
27	00:32:48	SELECT * FROM v_room_status LIMIT 0, 1000	16 row(s) returned

A **SELECT * FROM v_room_status** query to display the result of the created view.

Summary Query: Total Students per Room Type

```
57 SELECT rt.type_name, COUNT(s.status) AS n_occupants
58 FROM rooms r
59 JOIN room_types rt ON rt.type_id = r.type_id
60 LEFT JOIN students s ON s.room_id = r.room_id
61 WHERE s.status = 'ACTIVE'
62 GROUP BY rt.type_name;
63
```

type_name	n_occupants
Single	3
Double	8
Premium	3
Family	4

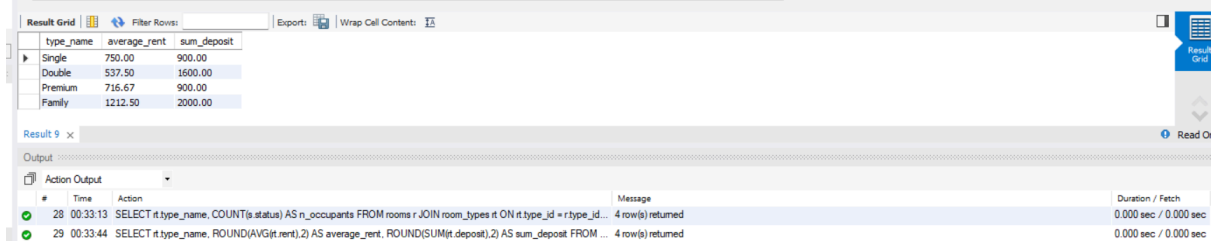
Output

#	Time	Action	Message
27	00:32:48	SELECT * FROM v_room_status LIMIT 0, 1000	16 row(s) returned
28	00:33:13	SELECT rt.type_name, COUNT(s.status) AS n_occupants FROM rooms r JOIN room_types rt ON rt.type_id = r.type_id...	4 row(s) returned

The **SELECT** query using **COUNT()** and **GROUP BY** to find the total number of active students categorized by their room type.

Summary Query: Avg Rent and Total Deposit

```
264 SELECT rt.type_name, ROUND(AVG(rt.rent),2) AS average_rent, ROUND(SUM(rt.deposit),2) AS sum_deposit
265 FROM rooms r
266 JOIN room_types rt ON rt.type_id = r.type_id
267 LEFT JOIN students s ON s.room_id = r.room_id
268 WHERE s.status = 'ACTIVE'
269 GROUP BY rt.type_name;
```



The screenshot shows a database interface with a query result grid. The grid has three columns: 'type_name', 'average_rent', and 'sum_deposit'. The data is as follows:

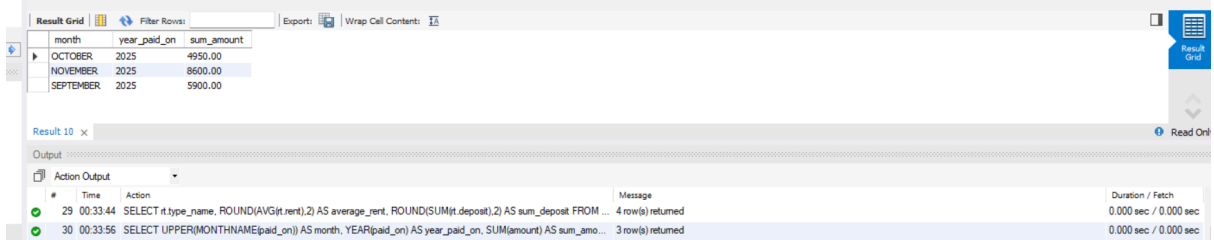
type_name	average_rent	sum_deposit
Single	750.00	900.00
Double	537.50	1600.00
Premium	716.67	900.00
Family	1212.50	2000.00

Below the grid, the 'Output' section shows the execution details for the query. It indicates that the query was executed at 00:33:44, returned 4 rows, and took 0.000 seconds to execute.

The SELECT query using ROUND(), AVG(), SUM(), and GROUP BY to calculate the average rent (rounded to 2 decimal places) and total deposit for each room type.

Summary Query: Monthly Payment Totals

```
271 SELECT UPPER(MONTHNAME(paid_on)) AS month, YEAR(paid_on) AS year_paid_on, SUM(amount) AS sum_amount
272 FROM payments
273 GROUP BY paid_on;
```



The screenshot shows a database interface with a query result grid. The grid has three columns: 'month', 'year_paid_on', and 'sum_amount'. The data is as follows:

month	year_paid_on	sum_amount
OCTOBER	2025	4950.00
NOVEMBER	2025	8600.00
SEPTEMBER	2025	5900.00

Below the grid, the 'Output' section shows the execution details for the query. It indicates that the query was executed at 00:33:56, returned 3 rows, and took 0.000 seconds to execute.

The SELECT query using UPPER(), MONTHNAME(), YEAR(), SUM(), and GROUP BY to total payments, grouped by the year and month.

Summary Query: OPEN Issues per Floor (HAVING)

```
275 SELECT r.floor_no, COUNT(m.status) AS total_issues
276 FROM maintenance m
277 JOIN rooms r ON r.room_id = m.room_id
278 WHERE status = 'OPEN'
279 GROUP BY floor_no
280 HAVING COUNT(m.status) > 2;
281
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	floor_no	total_issues		
▶	1	6		
	2	5		
	3	3		

Result 11 ×			
Output			
Action Output			
#	Time	Action	Message
30	00:33:56	SELECT UPPER(MONTHNAME(paid_on)) AS month, YEAR(paid_on) AS year_paid_on, SUM(amount) AS sum_amo...	3 row(s) returned
31	00:34:15	SELECT r.floor_no, COUNT(m.status) AS total_issues FROM maintenance m JOIN rooms r ON r.room_id = m.room_id ...	3 row(s) returned

The SELECT query using COUNT(), GROUP BY, and HAVING COUNT > 2 to report floors with more than two 'OPEN' maintenance issues.

Report Query: Room Rent Categorization

```
284 SELECT
285     CONCAT(UPPER(s.fname), ' ', UPPER(s.lname)) AS student_name,
286     ROUND(rent, 2) AS rent,
287     type_name AS room_type,
288     CASE
289         WHEN rent < 600 THEN 'LOW'
290         WHEN rent BETWEEN 600 AND 900 THEN 'MEDIUM'
291         ELSE 'HIGH'
292     END AS rent_category
293 FROM room_types rt
294 JOIN rooms r ON r.type_id = rt.type_id
295 LEFT JOIN students s ON s.room_id = r.room_id;
```

Result Grid

student_name	rent	room_type	rent_category
AHMAD AFIQ	700.00	Single	MEDIUM
FARID HASSAN	700.00	Single	MEDIUM
NADIA ZAINAL	750.00	Single	MEDIUM
HAFIZ ALI	750.00	Single	MEDIUM
AMIRAH OSMAN	800.00	Single	MEDIUM
ALIF FATHI	500.00	Double	LOW
SABRINA CARPENTER	500.00	Double	LOW
RIE TAKAHASHI	500.00	Double	LOW
IMAN ABADI	500.00	Double	LOW
BELLA ASTILLAH	500.00	Double	LOW
ADAM RAMLI	550.00	Double	LOW

Result 24 x

Output

Action Output

#	Time	Action	Message
45	00:42:20	SELECT CONCAT(s.fname, ' ', s.lname) AS student_name, ROUND(rent, 2) AS rent, type_name AS room_type, ...	26 row(s) returned
46	00:42:47	SELECT CONCAT(UPPER(s.fname), ' ', UPPER(s.lname)) AS student_name, ROUND(rent, 2) AS rent, type_na...	26 row(s) returned

The SELECT query applying CONCAT() and UPPER() to format the student's full name, and using ROUND() on the rent. It utilizes the CASE expression to categorize room rent as 'LOW' (rent < 600), 'MEDIUM' (rent between 600 and 900), or 'HIGH' (rent > 900).

Summary Table of Tables and Records

Table Name	Number of Records
room_types	12
rooms	16
students	26
maintenance	28
payments	26